CortexAI Microservices Architecture Documentation

This document explains the architecture, setup, files, functions, variables, and the detailed file-to-file state flow showing how the microservices are integrated and interact across CortexAI.

---

1. What is a Microservices Architecture? (For Beginners)

Instead of building a single giant backend (called a monolith), CortexAI is split into separate, focused mini-servers called **microservices**.
Each service:

1. Runs on its own network port (Gateway: 8000, Auth: 8001, Chat: 8002, Agent: 8003).
2. Has its own dedicated MongoDB database instance.
3. Performs a single set of tasks (Authentication, Chat History, or AI Agent processing).

To coordinate these services, a **Gateway** acts as a front door. The frontend client only talks to the Gateway (Port 8000), which checks permissions and routes requests to the correct service.

---

2. Services Breakdown: Variables & Functions

A. The Gateway (Port 8000)

- `protect` (middleware in `auth.middleware.js`):
  - **What it does**: Validates browser sessions.
  - **How it does it**: Extracts `session` cookie, verifies it against the Redis cache (`session-${sessionId}`), and sets `req.user`.

- `getCurrentuser` (controller in `user.controller.js`):
  - **What it does**: Directly returns `req.user` payload to resolve Gateway-level profile requests.
- `proxyWithHeader` (utility in `proxywithheader.js`):
  - **What it does**: Intercepts requests proxied downstream.
  - **How it does it**: Appends `x-user-id` (holding `req.user.userid`) into proxy headers so microservices know who the user is.

B. Auth Service (Port 8001)

- `login` (controller in `auth.controller.js`):
  - **What it does**: Authenticates Google users.
  - **How it does it**: Verifies JWT via Firebase Admin `verifyIdToken()`, creates a MongoDB `User` document if new, registers a session ID UUID in Redis, and sets a secure browser cookie.
- `logout` (controller in `auth.controller.js`):
  - **What it does**: Log out user sessions.
  - **How it does it**: Deletes session key from Redis and clears the client browser cookie.

C. CHAT Service (Port 8002)

- `createConversation` (controller in `chat.controller.js`):
  - **What it does**: Opens a new chat container.
  - **How it does it**: Reads `x-user-id` from header and creates a `Conversation` document.
- `getConversations` (controller in `chat.controller.js`):
  - **What it does**: Fetches the user's past chats list.
  - **How it does it**: Finds all conversations matching `userId` sorted by date descending.
- `saveMessage` (controller in `chat.controller.js`):
  - **What it does**: Saves individual message bubbles.
  - **How it does it**: Inserts a `Message` document storing `conversationId`, `role` (user/assistant), and `content`.
- `getMessage` (controller in `chat.controller.js`):
  - **What it does**: Retrieves the full message transcript of a chat.
  - **How it does it**: Returns all messages matching the `conversationId`.

- `updateConversation` (controller in `chat.controller.js`):
  - **What it does**: Renames chat titles.
  - **How it does it**: Updates `title` on the target `Conversation` document.

D. Agent Service (Port 8003)

- `agent` (controller in `agent.controller.js`):
  - **What it does**: Runs the multi-agent AI flow.
  - **How it does it**: Saves the user's prompt text to the CHAT service, invokes the LangGraph state compilation with the prompt, and returns the AI response.
- `chat` (node agent in `chat.agent.js`):
  - **What it does**: Handles general text inquiries.
  - **How it does it**: Invokes the default Groq LLM model and writes `aiResponse` to the state.
- `router` (node agent in `router.js`):
  - **What it does**: Route-intent classifier.
  - **How it does it**: Evaluates state context via Groq LLM to pick one of the active nodes (`chat`, `search`, `coding`, etc.).

---

3. Global Service Integration (Where & How They Are Used)

The microservices are interconnected across the codebase to process data:

A. Communication from Frontend to Backend

- **Authentication**:
  - *Where*: Called inside `home.jsx` and `logout.js`.
  - *How*: Fires HTTP requests to the Gateway on `/auth/login` (to write session cookies) and `/auth/logout` (to delete them).
- **Chat Navigation & Display**:
  - *Where*: Called inside `sidebar.jsx` and `chatarea.jsx`.
  - *How*: Hitting `/chat/create-conversation` (to add a new chat block) and

`/chat/get-conversation` (to show the list).

- **Prompt Submissions**:
  - *Where*: Called inside the prompt input box in `chatarea.jsx`.
  - *How*: Posts `{ prompt, conversationId }` payload to `/agent/chat` on the Gateway.

B. Inter-Service Communication (Backend to Backend)

- **Gateway to Microservices (Header Forwarding)**:
  - *Where*: Configured in `gateway/index.js`.
  - *How*: Requests to `/chat` and `/agent` are protected and proxied. The Gateway intercepts these requests and injects the `x-user-id` header using the `proxyWithHeader` utility.
- **Agent Service to CHAT Service (History Sync)**:
  - *Where*: Executed in `agent.controller.js` using `axios`.
  - *How*: Before starting the AI graph, the Agent Service calls `POST CHAT_SERVICE/save-message` to save the user's prompt into the MongoDB Chat database.

---

4. File-to-File Service Execution Workflow

This is the exact file path workflow when a user sends a message to the AI agent:

```
[Trigger] chatarea.jsx (Frontend)
   |  ——> User types prompt and clicks send.
   |  ——> Hits POST request to Gateway URL `/agent/chat`.
   ▼
[Route & Guard] gateway/index.js (Gateway)
   |  ——> Receives request and runs protect middleware in auth.middleware.js.
   |  ——> Reads Redis session data and sets user context.
   |  ——> Uses proxyWithHeader() to inject "x-user-id" header.
   |  ——> Proxies request to Agent Service on port 8003.
   ▼
[API Handler] agent/routes/route.js → agent.controller.js (Agent Service)
   |  ——> Receives prompt, conversationId, and user ID.
   |  ——> Calls POST CHAT_SERVICE/save-message to record user's prompt in Chat DB.
   |  ——> Invokes LangGraph: graph.invoke({ prompt, conversationId }).
   ▼
[State Compiler] graph/graph.js → graph/router.js
   |  ——> Router prompts the LLM inside config/llmmodel.js.
   |  ——> Returns the designated agent tag (e.g. "chat").
   ▼
[Agent Processor] agents/chat.agent.js
   |  ——> Invokes Groq LLM with prompt instructions.
   |  ——> Returns generated answer inside state as "aiResponse".
   ▼
[Response Handback] agent.controller.js
   |  ——> Receives Graph output, extracts aiResponse, and sends it back to Gateway.
   ▼
[Client Sync] chatarea.jsx
      ——> Receives response, updates Redux store, and renders message on screen.
```